ASSINGMENT 12--Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Task Description #1 (Sorting – Merge Sort Implementation)

Prompt: generate a Python program that implements the Merge Sort algorithm.

Input:

```
def merge_sort(arr):

    if len(arr) <= 1:
        return arr.copy()

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)



def merge(left, right):

    result = []
    i = 0
```

```
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result


# Test cases
numbers = [38, 27, 43, 3, 9, 82, 10]

print("Original list:", numbers)
print("Sorted list:", merge_sort(numbers))

print("Test 2:", merge_sort([5, 2, 8, 1]))
print("Test 3:", merge_sort([4, 4, 2, 2]))
print("Test 4:", merge_sort([]))
print("Test 5:", merge_sort([7]))
```

Explanation:

Merge Sort is a sorting algorithm that arranges elements in ascending or descending order. It uses the Divide and Conquer technique, which means dividing a problem into smaller parts, solving them, and combining the results.

Output:

```
PS C:\Users\Jeshwanth\.vscode\github> python -u "c:\Users\Jeshwanth\.vscode\github\12.py"
Original list: [64, 34, 25, 12, 22, 11, 90]
Sorted list(bubble sort):   [11, 12, 22, 25, 34, 64, 90]
Original list: [641, 342, 251, 142, 202, 131, 910]
Sorted list (Selection Sort): [131, 142, 202, 251, 342, 641, 910]
Original list: [41, 32, 51, 42, 20, 131, 9]
Sorted list (Insertion Sort): [9, 20, 32, 41, 42, 51, 131]
Original list: [4, 3, 25, 12, 2, 11, 9]
Sorted list (Quick Sort):    [2, 3, 4, 9, 11, 12, 25]
PS C:\Users\Jeshwanth\.vscode\github>
```

Task Description #2 (Searching – Binary Search with AI Optimization)

Prompt:

create a binary search function that finds a target element in a sorted list.

Input:

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    # Continue searching while the range is valid
    while low <= high:
        # Find the middle index
```

```
        mid = (low + high) // 2

        # Check whether the middle element is the target
        if arr[mid] == target:
            return mid

        # Search the right half if target is greater
        elif arr[mid] < target:
            low = mid + 1

        # Search the left half if target is smaller
        else:
            high = mid - 1

    # Return -1 if the target is not found
    return -1

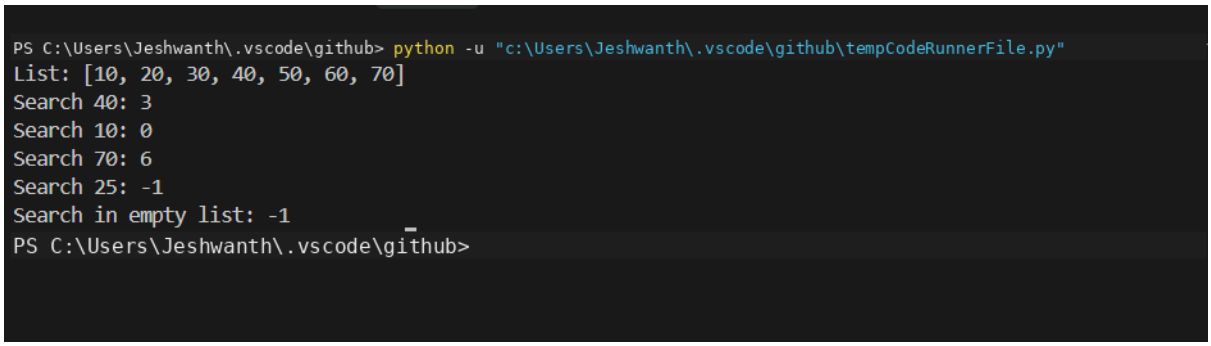

# Test cases
numbers = [10, 20, 30, 40, 50, 60, 70]

print("List:", numbers)
print("Search 40:", binary_search(numbers, 40))
print("Search 10:", binary_search(numbers, 10))
print("Search 70:", binary_search(numbers, 70))
print("Search 25:", binary_search(numbers, 25))
print("Search in empty list:", binary_search([], 10))
```

Explanation:

Binary Search finds a target element in a sorted list by repeatedly dividing the search range into two halves, returning its index if found or -1 otherwise.

Output:



Task Description #3 (Real-Time Application – Inventory Management System)

• Scenario: A retail store's inventory system contains

thousands of products, each with attributes like product ID,

name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Prompt:

o generate python code to suggest the most efficient search and sort

algorithms for this use case.

o Implement the recommended algorithms in Python.

o Justify the choice based on dataset size, update

frequency, and performance requirements.

Input:

Inventory Management System

```
inventory = [
    {"id": 101, "name": "Laptop", "price": 55000, "quantity": 10},
    {"id": 102, "name": "Mouse", "price": 500, "quantity": 50},
    {"id": 103, "name": "Keyboard", "price": 1200, "quantity": 30},
    {"id": 104, "name": "Monitor", "price": 12000, "quantity": 15},
    {"id": 105, "name": "Headphones", "price": 2000, "quantity": 25}
]

# Create a dictionary for fast product ID searching
product_index = {product["id"]: product for product in inventory}


def search_by_id(product_id):
    """Search by ID. Average time complexity: O(1)."""
    return product_index.get(product_id, "Product not found")


def search_by_name(name):
    """Search by name. Time complexity: O(n)."""
    for product in inventory:
        if product["name"].lower() == name.lower():
            return product
    return "Product not found"


def merge_sort(products, key):
    """
    Sort products in ascending order using Merge Sort.
```

```
    Time complexity: O(n log n).
    Auxiliary space complexity: O(n).
    """
    if len(products) <= 1:
        return products.copy()

    mid = len(products) // 2
    left = merge_sort(products[:mid], key)
    right = merge_sort(products[mid:], key)

    return merge(left, right, key)


def merge(left, right, key):
    """Merge two sorted product lists."""
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] <= right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
```

```
    return result



def display(products):
    """Display product details."""
    for product in products:
        print(product)



# Search operations
print("Search by ID:")
print(search_by_id(103))

print("\nSearch by Name:")
print(search_by_name("Laptop"))

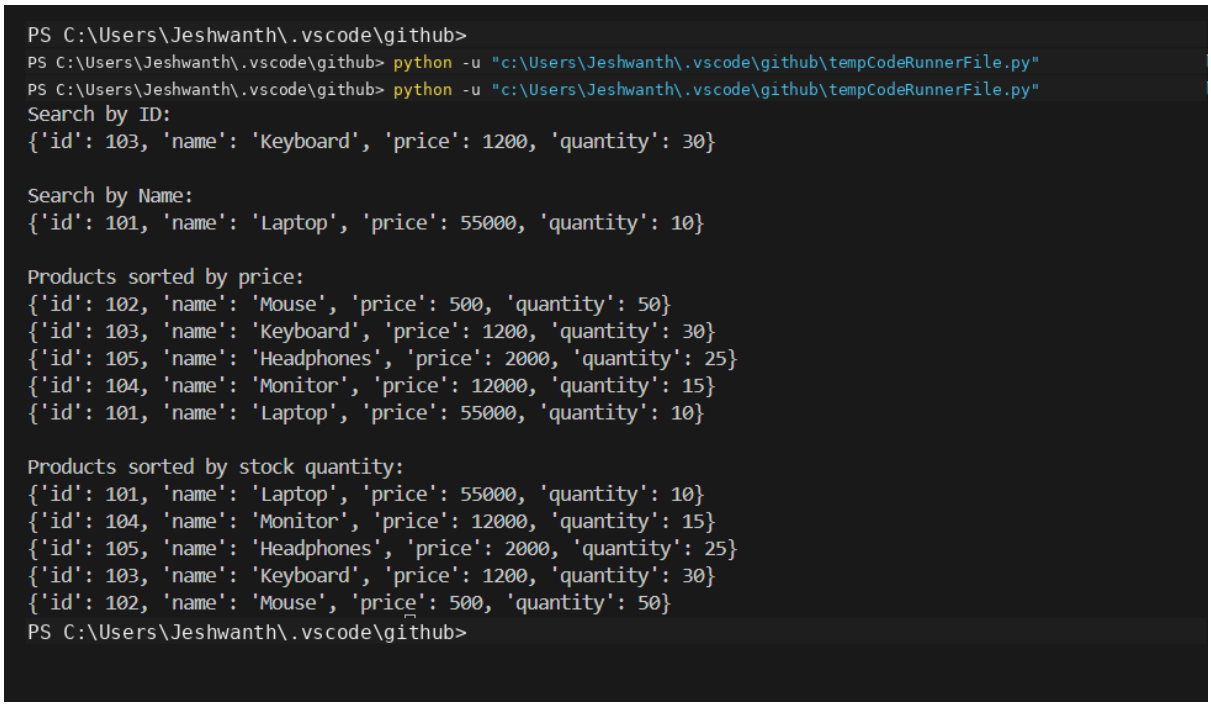
# Sorting operations
print("\nProducts sorted by price:")
display(merge_sort(inventory, "price"))

print("\nProducts sorted by stock quantity:")
display(merge_sort(inventory, "quantity"))
```

Explanation:

The Inventory Management System uses a dictionary for fast product ID searching, linear search for product names, and Merge Sort to arrange products by price or stock quantity efficiently.

Output:



Task description #4: Smart Hospital Patient Management System

A hospital maintains records of thousands of patients with details such as patient ID, name, severity level, admission date, and bill amount. Doctors and staff need to:

1. Quickly search patient records using patient ID or name.
2. Sort patients based on severity level or bill amount for prioritization and billing.

Student Task

Prompt

•generate a python code that recommend suitable searching and sorting algorithms.

• Justify the selected algorithms in terms of efficiency and suitability.

• Implement the recommended algorithms in Python.

Input:

```
patients = [
  {"id": 101, "name": "Ravi", "severity": 2, "admission_date": "2026-10-10", "bill": 15000},
  {"id": 102, "name": "Anjali", "severity": 1, "admission_date": "2026-10-11", "bill": 25000},
  {"id": 103, "name": "Kiran", "severity": 3, "admission_date": "2026-10-12", "bill": 10000},
  {"id": 104, "name": "Priya", "severity": 2, "admission_date": "2026-10-13", "bill": 30000}
]

# Fast patient ID search using a dictionary
patient_index = {p["id"]: p for p in patients}

def search_by_id(patient_id):
  return patient_index.get(patient_id, "Patient not found")

# Patient name search using Linear Search
def search_by_name(name):
  for patient in patients:
    if patient["name"].lower() == name.lower():
      return patient
  return "Patient not found"

# Merge Sort
def merge_sort(data, key):
  if len(data) <= 1:
    return data.copy()
```

```
    mid = len(data) // 2
    left = merge_sort(data[:mid], key)
    right = merge_sort(data[mid:], key)

    return merge(left, right, key)

def merge(left, right, key):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] <= right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Display patient records
print("Search by Patient ID:")
print(search_by_id(102))

print("\nSearch by Patient Name:")
```

```
print(search_by_name("Ravi"))


print("\nPatients sorted by Severity (highest priority first):")
display_severity = merge_sort(patients, "severity")
display_severity.reverse()
for patient in display_severity:
    print(patient)


print("\nPatients sorted by Bill Amount (lowest to highest):")
for patient in merge_sort(patients, "bill"):
    print(patient)
```

Explanation:

The Smart Hospital Patient Management System uses a dictionary for fast patient ID searches, linear search for patient names, and Merge Sort to arrange patients by severity level or bill amount efficiently.

Output:



Task Description #5: University Examination Result Processing System

A university processes examination results for thousands of students

containing roll number, name, subject, and marks. The system must:

1. Search student results using roll number.

2. Sort students based on marks to generate rank lists.

Prompt:

• generate a python code to Identify efficient searching and sorting algorithms

• Justify the choice of algorithms.

• Implement the algorithms in Python.

Input:

```
students = [
   {"roll_no": 101, "name": "Anjali", "subject": "Python", "marks": 85},
   {"roll_no": 102, "name": "Ravi", "subject": "Python", "marks": 92},
   {"roll_no": 103, "name": "Priya", "subject": "Python", "marks": 78},
   {"roll_no": 104, "name": "Kiran", "subject": "Python", "marks": 95}
]

# Search using Dictionary
student_index = {s["roll_no"]: s for s in students}

def search_student(roll_no):
   return student_index.get(roll_no, "Student not found")

# Sort using Merge Sort
def merge_sort(data):
```

```
    if len(data) <= 1:
        return data.copy()

    mid = len(data) // 2
    left = merge_sort(data[:mid])
    right = merge_sort(data[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i]["marks"] >= right[j]["marks"]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

print("Search Result:")
print(search_student(102))
```

```
print("\nRank List (Highest Marks First):")

rank_list = merge_sort(students)

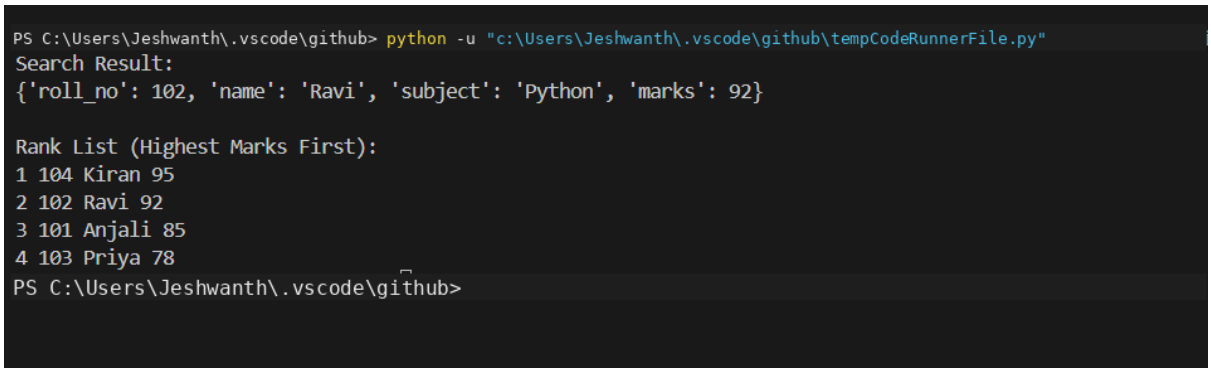

for rank, student in enumerate(rank_list, start=1):

    print(rank, student["roll_no"], student["name"], student["marks"])
```

Explanation:

The system uses a dictionary for fast roll-number searching and Merge Sort to generate a rank list from highest to lowest marks, with average search time \(O(1)\) and sorting time \(O(n\log n)\).

Output:



Task Description #6: Online Food Delivery Platform

An online food delivery application stores thousands of orders with

order ID, restaurant name, delivery time, price, and order status. The

platform needs to:

1. Quickly find an order using order ID.

2. Sort orders based on delivery time or price.

Prompt:

•generate a python code to suggest optimized algorithms.

• Justify the algorithm selection.

• Implement searching and sorting modules in Python.

Input:

```
orders = [
  {"order_id": 101, "restaurant": "Dominos", "delivery_time": 30, "price": 500, "status": "Delivered"},
  {"order_id": 102, "restaurant": "Burger King", "delivery_time": 20, "price": 300, "status": "Preparing"},
  {"order_id": 103, "restaurant": "Pizza Hut", "delivery_time": 40, "price": 700, "status": "On the way"},
  {"order_id": 104, "restaurant": "KFC", "delivery_time": 25, "price": 400, "status": "Delivered"}
]

# Fast search using Dictionary
order_index = {order["order_id"]: order for order in orders}

def search_order(order_id):
  return order_index.get(order_id, "Order not found")

# Merge Sort
def merge_sort(data, key):
  if len(data) <= 1:
    return data.copy()

  mid = len(data) // 2
  left = merge_sort(data[:mid], key)
  right = merge_sort(data[mid:], key)
```

```
    return merge(left, right, key)

def merge(left, right, key):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] <= right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Search an order
print("Search Order:")
print(search_order(102))

# Sort by delivery time
print("\nOrders sorted by delivery time:")
for order in merge_sort(orders, "delivery_time"):
    print(order)

# Sort by price
```

```
print("\nOrders sorted by price:")

for order in merge_sort(orders, "price"):

    print(order)
```

Explanation:

The system uses a dictionary to find orders quickly by order ID and Merge Sort to arrange orders by delivery time or price.

Output:

